ESP004 | PPGESP/IFBA 2026.2

Ambiente de Trabalho: Ubuntu 24.04 LTS + ROS 2 Jazzy Jalisco

Ferramenta: Jupyter Notebook/Lab com kernel ROS 2

Notebooks: NB-01 | NB-02 | NB-03 | NB-04

Prof. Dr. José Alberto Díaz Amado
Prof. Dr. João Erivando Soares Marques

PPGESP — Programa de Pós-Graduação em Engenharia e Sistemas e Projetos

Instituto Federal da Bahia — Campus Vitória da Conquista

2026.2

Contents

Como usar esta apostila **3**

1 Organização do Ambiente de Trabalho **4**
1.1 Por que Dual-Boot? . . . 4
1.1.1 Preparação do Windows 11 . . . 4
1.1.2 Particionamento do Disco . . . 5
1.2 Instalação do Ubuntu 24.04 LTS . . . 5
1.2.1 Criar pendrive bootável . . . 5
1.2.2 Instalação . . . 5
1.3 Instalação do ROS 2 Jazzy Jalisco . . . 5
1.4 Jupyter com Kernel ROS 2 . . . 6
1.5 Git e GitHub com SSH . . . 7

2 Fundamentos do ROS 2 Jazzy **9**
2.1 Nós e Ciclo de Vida . . . 9
2.2 Publisher e Timer . . . 10
2.3 Subscriber . . . 11
2.4 Parâmetros . . . 11
2.5 Cinemática Diferencial . . . 12
2.6 QoS — Qualidade de Serviço . . . 12
2.7 Exercício Guiado: QuadradoNode . . . 12

3 Jupyter Interativo e IA para Robótica **13**
3.1 Controle Interativo com ipywidgets . . . 13
3.2 Dashboard ao Vivo . . . 13
3.3 IA para Robótica: ObstacleNet . . . 14

4 ROS 2 com Google Colab e GitHub **15**

5 Atividade Avaliativa — PatrolNode **16**
5.1 Especificação do PatrolNode . . . 16
5.1.1 Máquina de Estados . . . 16
5.1.2 Parâmetros e Tópicos . . . 16
5.2 Rubrica de Avaliação . . . 17

A Apêndice A — Comandos ROS 2 de Referência **18**

B Apêndice B — Instalação de Dependências Python **18**

C Apêndice C — Estrutura de Repositório Sugerida **19**

Como usar esta apostila

Esta apostila é um **documento de referência ativo**: cada seção teórica possui um ou mais notebooks Jupyter correspondentes, indicados pela caixa laranja **[NB]**. A relação completa está na Tabela 1.

Table 1: Mapa Apostila ↔ Notebooks Jupyter

Seção	Título	Notebook	Células
1.1–1.6	Ambiente: Ubuntu, ROS 2, Jupyter, Git	NB-01	Cél. 1–6
2.1	Nós e ciclo de vida	NB-02	Cél. 1 (ros2 list)
2.2	Publisher + Timer (HelloNode)	NB-02	Cél. 2 (Ex. 2)
2.3	Subscriber (OuvidorNode)	NB-02	Cél. 3 (Ex. 3)
2.4	Parâmetros (VelocidadeNode)	NB-02	Cél. 4 (Ex. 4)
2.5	Cinemática Diferencial	NB-02	Cél. 5 (Ex. 5)
2.6	QoS	NB-02	Cél. 5 (nota QoS)
2.7	Exercício: QuadradoNode	NB-02	Cél. 6 (guiado)
3.1	Controle com ipywidgets	NB-03	Cél. 1–3
3.2	Dashboard ao vivo	NB-03	Cél. 4–5
3.3	IA: ObstacleNet (PyTorch)	NB-03	Cél. 6–9
4.1	ROS 2 com Google Colab	NB-04	Cél. de setup
5.1	Atividade: PatrolNode	NB-04	Cél. 1–7
5.2	Análise de dados	NB-04	Cél. 8–10
5.3	Entrega via Git	NB-04	Cél. 11

Como ler os símbolos

[NB] Existe código correspondente em um notebook Jupyter

[OK] Pré-requisito satisfeito / resultado esperado

[!] Atenção: passo crítico

[!!] Erro comum — leia com cuidado

1 Organização do Ambiente de Trabalho

Objetivo do Capítulo 1

Configurar o ambiente de desenvolvimento completo: dual-boot Ubuntu 24.04 + Windows 11, ROS 2 Jazzy instalado e sourced, Jupyter com kernel ROS 2, Git e GitHub com autenticacão SSH.

Notebook associado:

Notebook: NB-01 — ESP004_NB01_Verificacao_Ambiente.ipynb

Verifica automaticamente todos os componentes deste capítulo. Execute cada célula na ordem e observe os resultados **[OK]** / [FALHA].

→ nb01_ambiente_setup.ipynb

1.1 Por que Dual-Boot?

ROS 2 tem suporte nativo e completo apenas no Linux. A solução mais prática para laboratórios que já possuem Windows 11 instalado é o **dual-boot com GRUB**: ao ligar o computador, o aluno escolhe o sistema. Nenhuma virtualização é necessária, e a perda de desempenho é zero.

Por que não usar WSL 2?

O Windows Subsystem for Linux 2 não suporta adequadamente o middleware DDS do ROS 2 (especialmente multicast UDP), o que causa falhas na comunicação entre nós. Recomendamos o dual-boot para uso em laboratório.

1.1.1 Preparação do Windows 11

Antes de instalar o Ubuntu, desabilite:

1. **Fast Startup** (Inicialização Rápida): Painel de Controle → Opções de Energia → Escolher a função dos botões de liga/desliga → desmarcar "Ligar inicialização rápida".

2. **Secure Boot** (UEFI/BIOS): Reiniciar → tecla F2/Del/Esc (varia por fabricante) → Security → Secure Boot → Disabled.

3. **BitLocker** (se ativado): desativar antes de repartir o disco.

1.1.2 Particionamento do Disco

Espaço mínimo recomendado

```
/ (root):  40 GB ext4
swap:  4-8 GB (igual à RAM se ≤ 8 GB)
/home:  o restante (opcional)
```

No Windows: Gerenciamento de Disco → clicar com o botão direito na partição principal → "Diminuir Volume" → liberar pelo menos 50 GB.

1.2 Instalação do Ubuntu 24.04 LTS

1.2.1 Criar pendrive bootável

1. Baixar a ISO: ubuntu.com/download/desktop
2. Usar **Rufus** (Windows) ou **balenaEtcher**: modo GPT + UEFI, FAT32, sem MBR.
3. Reiniciar e pressionar F12/F11/Esc para o menu de boot.

1.2.2 Instalação

1. Selecionar "Instalação personalizada".
2. Criar partição `/` de 40 GB, tipo `ext4`, ponto de montagem `/`.
3. Selecionar o mesmo disco que o Windows como carregador de boot (GRUB será adicionado automáticamente).
4. Concluir a instalação e reiniciar.

1.3 Instalação do ROS 2 Jazzy Jalisco

ROS 2 Jazzy Jalisco

Distribuição LTS do ROS 2 para Ubuntu 24.04. Suporte até **maio de 2029**. Ideal para uso acadêmico e projetos de longa duração.

[NB-01] Célula 2 verifica se o ROS 2 está corretamente instalado e sourced.

Listing 1: Instalação do ROS 2 Jazzy (executar no terminal)

```
# 1. Locale
sudo apt update && sudo apt install -y locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
```

```
5
6  # 2. Repositorio OSRF
7  sudo apt install -y software-properties-common curl
8  sudo curl -sSL https://raw.githubusercontent.com/ros/
       rosdistro/master/ros.key \
9    -o /usr/share/keyrings/ros-archive-keyring.gpg
10 echo "deb [arch=$(dpkg --print-architecture) \
11 signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \
12 http://packages.ros.org/ros2/ubuntu \
13 $(. /etc/os-release && echo $UBUNTU_CODENAME) main" \
14   | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
15
16 # 3. Instalacao
17 sudo apt update
18 sudo apt install -y ros-jazzy-desktop python3-colcon-common-
       extensions
19
20 # 4. Source automatico no bashrc
21 echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc
22 echo "export ROS_DOMAIN_ID=0" >> ~/.bashrc
23 source ~/.bashrc
```

1.4 Jupyter com Kernel ROS 2

O Jupyter só enxerga o `rclpy` se o ambiente ROS 2 for carregado **antes** de iniciar o kernel Python. O truque é criar um kernel customizado que faça isso automaticamente.

[NB-01] Células 1–3 verificam se o Jupyter e o kernel estão corretos.

Listing 2: Instalar Jupyter e criar kernel ROS 2

```
1  # Instalar Jupyter Lab e ipykernel
2  pip3 install --break-system-packages jupyterlab ipykernel
       ipywidgets
3
4  # Criar diretorio do kernel
5  mkdir -p ~/.local/share/jupyter/kernels/ros2jazzy
6
7  # Criar o arquivo kernel.json
8  cat > ~/.local/share/jupyter/kernels/ros2jazzy/kernel.json <<
       'EOF'
9  {
10   "argv": [
```

```
    "bash", "-c",
    "source␣/opt/ros/jazzy/setup.bash␣&&␣exec␣python3␣-m␣
       ipykernel_launcher␣-f␣{connection_file}"
  ],
  "display_name": "Python␣3␣(ROS2␣Jazzy)",
  "language": "python",
  "env": {
    "ROS_DOMAIN_ID": "0"
  }
}
EOF

# Verificar
jupyter kernelspec list
```

Dica — Kernel correto

Ao abrir qualquer notebook da disciplina, selecione sempre o kernel **"Python 3 (ROS2 Jazzy)"** no canto superior direito. Se usar o kernel padrão do Python, o `import rclpy` vai falhar.

1.5 Git e GitHub com SSH

[NB-01] Célula 4 verifica a configuração do Git e a chave SSH.

Listing 3: Configurar identidade e chave SSH para GitHub

```
# Identidade global
git config --global user.name  "Seu␣Nome␣Completo"
git config --global user.email "seu.email@ifba.edu.br"
git config --global init.defaultBranch main

# Gerar chave SSH Ed25519 (mais segura que RSA)
ssh-keygen -t ed25519 -C "seu.email@ifba.edu.br" -f ~/.ssh/
   id_ed25519_github

# Exibir a chave publica para adicionar no GitHub
cat ~/.ssh/id_ed25519_github.pub

# Testar autenticacao
ssh -T git@github.com
```

Adicione a chave em: github.com/settings/keys

Conventional Commits — obrigatório

Todos os commits devem seguir o padrão:

```
<tipo>: <descricao curta em minusculas>
```

Tipos: `feat:` `fix:` `docs:` `refactor:` `test:`

2 Fundamentos do ROS 2 Jazzy

Objetivo do Capítulo 2

Compreender e implementar os conceitos fundamentais do ROS 2: nós, tópicos (publisher/subscriber), serviços, parâmetros e QoS. Todos os exemplos são executados diretamente no Jupyter com o kernel ROS 2.

Notebooks associados:

Notebook: NB-02 — ESP004_NB02_Fundamentos_ROS2.ipynb

5 exemplos progressivos + 1 exercício guiado. Abrir e executar em ordem. Cada exemplo corresponde a uma subseção abaixo.

→ nb02_ros2_fundamentos.ipynb

2.1 Nós e Ciclo de Vida

[NB-02] Célula 1: inspecionar o sistema ROS 2 com comandos de linha.

O que é um Nó?

Um **nó** (node) é um processo ROS 2 com uma identidade (`/nome`). Nós se comunicam trocando mensagens por **tópicos** ou chamando **serviços**. Em Python, herdam de `rclpy.node.Node`.

Listing 4: Estrutura mínima de um nó ROS 2

```python
import rclpy
from rclpy.node import Node

class MeuNo(Node):
    def __init__(self):
        super().__init__('meu_no')       # /meu_no no grafo ROS
        self.get_logger().info('No iniciado!')

rclpy.init()
no = MeuNo()
rclpy.spin_once(no, timeout_sec=1.0)
no.destroy_node()
rclpy.shutdown()
```

2.2 Publisher e Timer

[NB-02] Célula 2 (Exemplo 2): `HelloNode` publica uma `std_msgs/String` a cada 1 segundo por 5 segundos.

Listing 5: HelloNode — Publisher com Timer (NB-02 Ex.2)

```python
import rclpy, threading
from rclpy.node import Node
from std_msgs.msg import String

class HelloNode(Node):
    def __init__(self):
        super().__init__('hello_node')
        self.pub   = self.create_publisher(String, '/esp004/
           hello', 10)
        self.timer = self.create_timer(1.0, self._cb)
        self.count = 0

    def _cb(self):
        msg = String()
        msg.data = f'Ola␣do␣Jupyter!␣#{self.count}'
        self.pub.publish(msg)
        self.get_logger().info(msg.data)
        self.count += 1

# --- Threading: essencial no Jupyter para nao travar o
   kernel ---
rclpy.init()
no = HelloNode()
stop = threading.Event()

def spin():
    while not stop.is_set():
        rclpy.spin_once(no, timeout_sec=0.05)

t = threading.Thread(target=spin, daemon=True)
t.start()
import time; time.sleep(5)
stop.set(); t.join()
no.destroy_node(); rclpy.shutdown()
```

Por que threading?

`rclpy.spin()` bloqueia o kernel Jupyter indefinidamente. O padrão adotado na disciplina usa um **daemon thread** com `spin_once()` em loop, controlado por um `threading.Event()`.

2.3 Subscriber

[NB-02] Célula 3 (Exemplo 3): `OuvidorNode` recebe mensagens do tópico `/esp004/hello`.

Listing 6: OuvidorNode — Subscriber (NB-02 Ex.3)

```python
from rclpy.node import Node
from std_msgs.msg import String

class OuvidorNode(Node):
    def __init__(self):
        super().__init__('ouvidor_node')
        self.sub = self.create_subscription(
            String, '/esp004/hello', self._cb, 10)
        self.mensagens = []

    def _cb(self, msg):
        self.get_logger().info(f'Recebi: {msg.data}')
        self.mensagens.append(msg.data)
```

2.4 Parâmetros

[NB-02] Célula 4 (Exemplo 4): `VelocidadeNode` declara parâmetros de velocidade alteraveis em tempo de execução.

Listing 7: Parâmetros dinâmicos (NB-02 Ex.4)

```python
class VelocidadeNode(Node):
    def __init__(self):
        super().__init__('velocidade_node')
        self.declare_parameter('vel_linear',  0.3)
        self.declare_parameter('vel_angular', 0.5)

    def get_velocidades(self):
        vl = self.get_parameter('vel_linear').value
        va = self.get_parameter('vel_angular').value
        return vl, va

```

```
# Alterar em tempo de execucao:
# no.set_parameters([Parameter('vel_linear', value=0.8)])
```

2.5 Cinemática Diferencial

[NB-02] Célula 5 (Exemplo 5): simulador com gráfico de trajetória (3 paineis: XY, θ, distância).

Modelo cinemático diferencial

$$x_{k+1} = x_k + v\cos(\theta_k)\,\Delta t$$

$$y_{k+1} = y_k + v\sin(\theta_k)\,\Delta t$$

$$\theta_{k+1} = \theta_k + \omega\,\Delta t$$

onde v é a velocidade linear [m/s] e ω a angular [rad/s].

2.6 QoS — Qualidade de Serviço

A Tabela 2 resume as políticas de QoS mais usadas em robótica.

Table 2: Políticas de QoS e aplicações típicas

Política	**Opções**	**Uso típico**
Confiabilidade	RELIABLE / BEST_EFFORT	Cmds críticos / sensores
Durabilidade	TRANSIENT_LOCAL / VOLATILE	Status do robô / stream
Histórico	KEEP_LAST (N) / KEEP_ALL	Fila limitada / ilimitada

2.7 Exercício Guiado: QuadradoNode

[NB-02] Célula 6: implementar um robô que percorre um quadrado usando uma máquina de estados simples.

Exercício — QuadradoNode

Estados: `FRENTE` → `CURVA` → `FRENTE` → ... (4 vezes)

Parâmetros: $v = 0.3$ m/s, $\omega = 0.785$ rad/s (45°/s ⇒ 2 s para girar 90°).

3 Jupyter Interativo e IA para Robótica

Objetivo do Capítulo 3

Usar o Jupyter como interface de controle em tempo real e integrar redes neurais PyTorch ao pipeline ROS 2.

Notebook: NB-03 — ESP004_NB03_Jupyter_Interativo.ipynb

3 seções: controle com widgets, dashboard ao vivo e treinamento da ObstacleNet.

→ nb03_jupyter_ros2_interativo.ipynb

3.1 Controle Interativo com ipywidgets

[NB-03] Células 1–3: painel com dois sliders e botão de parada emergencial. O painel publica `geometry_msgs/Twist` em tempo real.

Listing 8: Painel de controle com ipywidgets (NB-03)

```
import ipywidgets as widgets

sl_vlin = widgets.FloatSlider(
    value=0.0, min=-0.5, max=0.5, step=0.05,
    description='Linear [m/s]')
sl_vang = widgets.FloatSlider(
    value=0.0, min=-1.5, max=1.5, step=0.1,
    description='Angular [rad/s]')
btn_stop = widgets.Button(
    description='PARAR', button_style='danger')

def on_slide(change):
    msg.linear.x  = sl_vlin.value
    msg.angular.z = sl_vang.value
    pub.publish(msg)

sl_vlin.observe(on_slide, names='value')
sl_vang.observe(on_slide, names='value')
```

3.2 Dashboard ao Vivo

[NB-03] Células 4–5: simulação de 8 segundos com gráfico atualizado a cada 0,5 s usando `clear_output(wait=True)`.

3.3 IA para Robótica: ObstacleNet

[NB-03] Células 6–9: rede neural MLP para detecção de obstáculos treinada e integrada ao ROS 2.

Arquitetura ObstacleNet

$$\text{Entrada}(4) \rightarrow \text{Linear}(32) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.2) \rightarrow \text{Linear}(16) \rightarrow \text{ReLU} \rightarrow \text{Linear}(1) \rightarrow \text{Sigmoid}$$

Entrada: $[v_x,\ v_y,\ d_{\text{scan}},\ \omega]$ Saída: $P(\text{obstáculo}) \in [0, 1]$

Listing 9: ObstacleNet em PyTorch (NB-03)

```
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(4, 32), nn.ReLU(),
    nn.Dropout(0.2),
    nn.Linear(32, 16), nn.ReLU(),
    nn.Linear(16, 1),  nn.Sigmoid()
)
# Treinamento: 60 epocas, Adam lr=1e-3, BCELoss
# Salvar: torch.save(model.state_dict(), 'models/
    obstacle_net_v1.pt')
```

4 ROS 2 com Google Colab e GitHub

Objetivo do Capítulo 4

Usar o Google Colab como alternativa quando não há acesso ao Ubuntu local. Integrar notebooks Colab ao GitHub via tokens seguros.

Notebook: NB-04 (Etapa de Setup) — Colab + GitHub

O NB-04 inclui células de configuração para rodar o ambiente Colab com `rosbags` e PyTorch.

→ nb04_atividade_avaliativa.ipynb

Listing 10: Instalar ROS 2 e rosbags no Colab

```
# Celula de setup (Colab)
import subprocess, sys

def run(cmd):
    subprocess.run(cmd, shell=True, check=True)

run("pip install rosbags torch numpy matplotlib pandas
   ipywidgets -q")
# rclpy nao disponivel no Colab --- usar rosbags para leitura
    de bags
from rosbags.rosbag2 import Reader
```

5 Atividade Avaliativa — PatrolNode

Objetivo do Capítulo 5

Implementar o **PatrolNode**: um robô de patrulha com máquina de estados que percorre um quadrado e desvia de obstáculos. Entregar via Git com mínimo de 5 commits.

Notebook: NB-04 — ESP004_NB04_Atividade_Avaliativa.ipynb

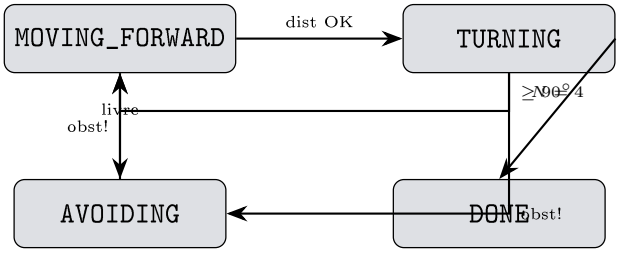
Documento completo da atividade avaliativa. Preencha as células de identificação e implemente o PatrolNode conforme especificado.

→ nb04_atividade_avaliativa.ipynb

5.1 Especificação do PatrolNode

[NB-04] Células 1–7: implementação guiada.

5.1.1 Máquina de Estados



5.1.2 Parâmetros e Tópicos

Table 3: Parâmetros do PatrolNode

Parâmetro	Valor	Descrição
`VEL_LINEAR`	0.3 m/s	Velocidade de avanço
`VEL_ANGULAR`	0.785 rad/s	Velocidade de giro (45°/s)
`DIST_LADO`	2.0 m	Comprimento de cada lado
`N_LADOS`	4	Número de lados (quadrado)

Table 4: Tópicos do PatrolNode

Tópico	Tipo	Direção
`/cmd_vel`	`geometry_msgs/Twist`	Publicar
`/obstaculo`	`std_msgs/Bool`	Assinar

5.2 Rubrica de Avaliação

Table 5: Rubrica — 100 pontos

Critério	**Pontos**
Git com ≥ 5 commits (Conventional Commits)	15
PatrolNode: máquina de estados completa	35
Lógica de desvio de obstáculo	20
Análise de dados + gráficos matplotlib	20
Código limpo + docstrings	10
Total	**100**

A Apêndice A — Comandos ROS 2 de Referência

Listing 11: Comandos ROS 2 essenciais

```
# Listar nos ativos
ros2 node list

# Inspecionar um no
ros2 node info /nome_do_no

# Listar topicos
ros2 topic list

# Ler mensagens de um topico
ros2 topic echo /esp004/hello

# Publicar manualmente
ros2 topic pub /esp004/hello std_msgs/String '{data: "teste"}
    '

# Listar servicos
ros2 service list

# Inspecionar parametros
ros2 param list /nome_do_no
ros2 param get  /nome_do_no nome_param
ros2 param set  /nome_do_no nome_param 0.8

# Gravar e reproduzir bags
ros2 bag record /cmd_vel /obstaculo -o minha_bag
ros2 bag play   minha_bag
ros2 bag info   minha_bag
```

B Apêndice B — Instalação de Dependências Python

Listing 12: Instalar todas as dependências Python

```
pip3 install --break-system-packages \
    numpy matplotlib pandas \
    torch torchvision \
    rosbags \
```

```
    ipywidgets
```

C Apêndice C — Estrutura de Repositório Sugerida

Listing 13: Estrutura de repositório para a atividade avaliativa

```
esp004-seu-nome/
  notebooks/
    nb01_ambiente_setup.ipynb
    nb02_ros2_fundamentos.ipynb
    nb03_jupyter_ros2_interativo.ipynb
    nb04_atividade_avaliativa.ipynb
  models/
    obstacle_net_v1.pt
    scaler_config.json
  bags/
    .gitkeep
  .gitignore
  README.md
```

References

[1] Open Robotics. *ROS 2 Jazzy Jalisco Documentation*. https://docs.ros.org/en/jazzy/, 2024.

[2] Canonical Ltd. *Ubuntu 24.04 LTS (Noble Numbat) Release Notes*. https://ubuntu.com/blog/tag/ubuntu-24-04, 2024.

[3] Open Robotics. *rclpy — ROS 2 Python Client Library*. https://github.com/ros2/rclpy, 2024.

[4] Project Jupyter. *JupyterLab Documentation*. https://jupyterlab.readthedocs.io/, 2024.

[5] A. Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Advances in Neural Information Processing Systems, 2019.

[6] B. Siciliano et al. *Robotics: Modelling, Planning and Control*. Springer, 2009.